Dossier de prompts — Module Caméras (à exécuter après la refonte)

Prompts pour ChatGPT · version 1.0 · 31 octobre 2026 · travail sur main, sans branche, PR, CI ni déploiement sans accord explicite

À quoi sert ce dossier

Dernier ajout : un **module Caméras** pour voir ses caméras de surveillance depuis l'application et relier les alertes aux images. À exécuter **uniquement après tous les autres prompts et après la refonte du design (série 8)**, pour reprendre le design final.

Applications concernées

Application	Intérêt principal
Kiosko	Comptoir et caisse du point mobile money
Bootik	Caisse, réserve, dépôt de gaz
Zipple	Caisse, bar, salle, anti-fraude
NAXA Services	Caisse, dépôt et retrait du pressing
Loyable	Entrées d'immeubles, parking, parties communes
NAXA Hotel	Réception, accès, caisse ; jamais les chambres

CotiZ n'est pas concernée : il n'y a pas de lieu à surveiller. L'application des écoles réutilisera le même module.

Ordre d'exécution

1. 9.0 Module commun (une seule fois, d'abord).
2. 9.1 à 9.6 : une intégration par application.
3. 9.7 Vision de nuit, détection de mouvement et alertes IA.
4. 9.8 Recette (en dernier).

Mode d'emploi

1. Ouvrez une conversation ChatGPT neuve pour chaque prompt (ou continuez la même conversation si elle reste courte). Collez le prompt en entier.
2. Dans le prompt, remplacez « [COLLE ICI LE FICHIER DE PASSATION] » par le fichier de passation produit à la fin de l'étape précédente. Pour le tout premier prompt (0.1), il n'y en a pas.
3. Un seul prompt à la fois. Ne passez au suivant qu'après avoir relu le résultat, fait les contrôles locaux indiqués et donné votre accord par écrit.
4. À chaque réponse, ChatGPT doit finir par un fichier de passation et par la phrase « Étape terminée. J'attends ton accord… ». Conservez ce fichier : c'est la mémoire du projet.

Règles de travail à respecter à chaque étape

1. Travaille uniquement sur la branche main. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

Contrôles après chaque étape

- Le travail est bien sur main, sans branche ni pull request créées.
- Aucun workflow CI lancé, aucun déploiement, aucune migration appliquée sans votre accord.
- Les textes sont dans les fichiers FR/EN ; aucun PIN ni OTP n'apparaît dans le code, les journaux ou les messages.
- Les tests demandés par le prompt passent en local ; les calculs d'argent sont exacts au franc près.
- Le fichier de passation est complet (7 rubriques) et sauvegardé.

Les prompts

Chaque prompt ci-dessous est autonome : copiez-le en entier dans ChatGPT, en remplaçant [COLLE ICI LE FICHIER DE PASSATION] par le fichier de l'étape précédente.

PROMPT 9.0 — Module Caméras commun

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Construire le module de vidéosurveillance partagé par les 6 applications, du plus simple au plus complet.

À FAIRE :

1. Trois modes de connexion, du plus facile au plus complet : (1) bouton qui ouvre l'application de la marque (Hik-Connect, EZVIZ, Dahua ou Imou, Reolink, TP-Link Tapo…) par lien profond, gratuit et immédiat ; (2) vue en direct dans l'application sur le réseau local via RTSP ou ONVIF avec un lecteur libre (par exemple ExoPlayer ou libVLC : compare, choisis, justifie) ; (3) accès à distance par la marque (P2P) ou par un relais que le client installe (passerelle libre type MediaMTX ou go2rtc sur un petit ordinateur ou un téléphone Android, avec un tunnel gratuit : vérifie les conditions et les limites actuelles avant de recommander).
2. Assistant de connexion : 1) choisir la marque ou « autre caméra » ; 2) scanner le QR de la caméra ou chercher les caméras du réseau (découverte ONVIF) ou saisir l'adresse ; 3) tester la connexion ; 4) nommer la caméra et son emplacement ; 5) choisir qui peut la voir. Fournis des modèles d'adresse de flux par marque comme AIDE À PRÉ-REMPLIR (principe : Hikvision rtsp://identifiant:mot-de-passe@adresse:554/Streaming/Channels/101 pour le flux principal et /102 pour le secondaire ; Dahua .../cam/realmonitor?channel=1&subtype=0 ou 1 ; Reolink .../h264Preview_01_main ou _sub ; Tapo .../stream1 ou stream2) en précisant que chacun est à VÉRIFIER dans la documentation de la marque avant usage.
3. Écrans : liste des caméras par site (avec état en ligne ou hors ligne), vue d'une caméra, mosaïque 2 sur 2, mode basse consommation, capture d'image enregistrée sur l'appareil, partage de la capture à un incident ou un ticket, plein écran.
4. Lien alerte vers images : depuis une alerte de l'application (écart de caisse, annulation ou remise suspecte, écart de stock, incident), un bouton « Voir la caméra » ouvre la bonne caméra à l'heure de l'événement : relecture si la caméra ou l'enregistreur la permet (par la marque ou ONVIF), sinon vue en direct et rappel de l'heure pour la relecture dans l'application de la marque. Précise honnêtement ce qui est possible selon le matériel.
5. Droits et journal : qui voit quelle caméra par poste ; journal de chaque consultation ; désactivation immédiate d'un accès ; aucune donnée vidéo stockée dans Supabase (seules des métadonnées : nom, emplacement, droits, journal).
6. Paliers : Essentiel = bouton vers l'application de la marque et 1 caméra ; Pro = vue en direct dans l'application, mosaïque, captures, jusqu'à 8 caméras ; Réseau = multi-sites, accès à distance par relais, liens alerte vers images, journal complet, caméras illimitées. [P : à confirmer par moi]
7. Vision de nuit, détection de mouvement et alertes IA : voir le prompt 9.7, à exécuter après les intégrations par application. Dans ce prompt, prévois seulement les points d'extension (table des événements, notifications critiques avec capture d'image, règles par caméra).

CONTRÔLES (en local uniquement, aucune CI) : Démonstration avec une caméra simulée (flux de test RTSP local) ; assistant testé de bout en bout ; aucun identifiant visible dans les journaux ni dans Supabase.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.

- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.1 — Intégration dans Kiosko

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final

de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Brancher le module Caméras dans Kiosko.

À FAIRE :

1. Emplacements typiques : comptoir et caisse du point mobile money, porte. Associe chaque caméra à un point de vente.
2. Alertes reliées : écart de caisse à la clôture, trou de solde non résolu, annulation ou correction d'opération, opération « à vérifier » : bouton « Voir la caméra du comptoir » à l'heure de l'événement.
3. Écran Santé de l'appli : ajoute l'état des caméras (en ligne ou hors ligne) ; notification si une caméra tombe hors ligne plus de 15 minutes.

CONTRÔLES (en local uniquement, aucune CI) : Test : depuis un écart de caisse, la bonne caméra s'ouvre avec l'heure affichée.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.

6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.2 — Intégration dans Bootik

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Brancher le module Caméras dans Bootik.

À FAIRE :

1. Emplacements typiques : caisse, rayons, réserve, zone du dépôt de gaz, entrée. Une caméra peut couvrir plusieurs zones.
2. Alertes reliées : écart de caisse, annulation ou remise forte, écart d'inventaire (stock théorique contre comptage), échange de bouteille de gaz inhabituel, retour de marchandise.
3. Multi-boutiques : le compte mère voit la mosaïque de ses boutiques selon ses droits ; rapport mensuel des incidents avec captures.

CONTRÔLES (en local uniquement, aucune CI) : Test : depuis un écart d'inventaire, ouverture de la caméra de la réserve ; droits par poste respectés.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.

- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.3 — Intégration dans Zipple

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Brancher le module Caméras dans Zipple.

À FAIRE :

1. Emplacements typiques : caisse, bar, cuisine (zone de réception de marchandises), salle et terrasse, entrée ; pas de caméra dans les toilettes.
2. Alertes reliées : annulations répétées, offerts anormaux, écart de caisse par serveur, écart de stock théorique contre réel, remises non validées : bouton « Voir la caméra du bar ou de la caisse » à l'heure de l'événement.

3. Mode soirée : vue rapide en une touche de la caisse et de la salle depuis l'écran d'accueil ; flux secondaire basse définition pour économiser les données.

CONTRÔLES (en local uniquement, aucune CI) : Test : depuis une annulation suspecte, ouverture de la caméra de la caisse à l'heure de l'annulation.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.4 — Intégration dans NAXA Services

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Brancher le module Caméras dans NAXA Services.

À FAIRE :

1. Emplacements typiques : caisse, accueil, zone de dépôt et de retrait du pressing (preuve de remise des vêtements), salle d'attente ; pas de caméra dans les zones d'essayage ni d'intimité.
2. Alertes reliées : écart de caisse, annulation de prestation, vêtement non retiré (aide à retrouver une remise), litige client : capture jointe à la fiche de dépôt.
3. Vie privée des clientes et clients : rappel d'affichage obligatoire et paramètre pour masquer certaines caméras aux employés.

CONTRÔLES (en local uniquement, aucune CI) : Test : capture attachée à un dépôt de pressing ; droits par poste.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.5 — Intégration dans Loyable

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Brancher le module Caméras dans Loyable.

À FAIRE :

1. Emplacements typiques : entrée d'immeuble, cour, parking, couloir commun ; jamais l'intérieur des logements ni les paliers privés.
2. Association des caméras aux biens et aux immeubles ; vue par immeuble pour un gestionnaire de plusieurs propriétaires, avec droits par propriétaire (un propriétaire ne voit jamais les caméras d'un autre).
3. Alertes reliées : incident ou dégradation signalé sur un bien (capture jointe), travaux et accès des prestataires, état des lieux de sortie (captures des parties communes).

CONTRÔLES (en local uniquement, aucune CI) : Test : isolation des caméras entre propriétaires ; capture jointe à un incident.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.

- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.6 — Intégration dans NAXA Hotel

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Brancher le module Caméras dans NAXA Hotel.

À FAIRE :

1. Emplacements typiques : réception, couloirs, entrées, parking, caisse, bar et restaurant, zones techniques ; JAMAIS les chambres ni les salles de bains.
2. Droits par poste : propriétaire et directeur voient tout le groupe ; manager son hôtel ; chef de réception et agent de sécurité les caméras d'accès et de réception ; femme de chambre et autres postes : aucune par défaut.
3. Alertes et incidents reliés : fiche d'incident avec capture, litige à la réception, écart de caisse, remise ou annulation de note, clôture de nuit (vue rapide des caméras d'accès pour le réceptionniste de nuit).
4. Multi-hôtels : mosaïque d'une caméra clé par hôtel pour la direction, avec journal d'accès ; tout cela reste optionnel et activé hôtel par hôtel.

CONTRÔLES (en local uniquement, aucune CI) : Test : un poste sans droit ne voit aucune caméra ; fiche d'incident avec capture ; journal d'accès complet.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.7 — Vision de nuit, détection de mouvement et alertes IA

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Permettre de voir la nuit, de détecter les événements et d'alerter le gestionnaire, du niveau le plus simple au plus puissant.

À FAIRE :

1. Guide de choix des caméras (écran d'aide et document) : vision nocturne infrarouge ou basse lumière (portée adaptée à la zone, image en noir et blanc de nuit), détection intégrée de personne ou de véhicule, prise en charge des événements ONVIF, alimentation stable et onduleur. L'assistant de connexion teste ce que la caméra sait faire (flux, instantané, événements) et le dit clairement. N'impose aucune marque et n'invente aucun prix.
2. Affichage de nuit : le flux infrarouge s'affiche comme un flux normal ; ajoute un mode « nuit » (contraste et luminosité réglables, éclaircissement léger de l'image) en prévenant que le logiciel ne crée pas de détails absents de l'image.
3. NIVEAU 1, événements de la caméra : guide pas à pas, par marque, pour activer les notifications de détection de la marque ; dans notre application, bouton « Voir la caméra » et raccourci vers l'application de la marque. Gratuit, aucun matériel en plus.
4. NIVEAU 2, passerelle sur le site : petit composant (petit ordinateur, mini-PC ou téléphone Android toujours allumé) qui reçoit les événements de la caméra ou de l'enregistreur (ONVIF ou notification), prend un instantané, le compresse et l'envoie au serveur par une fonction sécurisée (jeton, aucun port exposé) ; configuration guidée, surveillance de son état (en ligne ou hors ligne) et notification si la passerelle tombe.
5. Règles d'alerte par caméra, zone et horaire : type (personne, véhicule, mouvement), durée minimale, plage horaire (fermeture, nuit, week-end) avec exceptions, gravité (information, importante, critique). Exemples : présence après la fermeture, porte ouverte la nuit, véhicule au parking après une heure donnée. Une alerte critique envoie une notification avec la capture d'image et les boutons « Voir la caméra », « Appeler le contact » et « Prévenir le vigile par WhatsApp » (message préparé).
6. Fausses alertes : sensibilité réglable, regroupement des événements répétés (une alerte par rafale sur une fenêtre réglable), plages de silence, bouton « utile » ou « inutile » sur chaque alerte, statistiques du taux d'alertes utiles par caméra, suggestion de réglage. Objectif du pilote : au moins 80 % d'alertes jugées utiles. [P]
7. NIVEAU 3, IA sur place (optionnel, palier Réseau) : rédige d'abord un rapport de faisabilité (modèle libre de détection de personnes et de véhicules, licence, matériel nécessaire, charge, consommation, coût estimé à vérifier) et attends mon accord avant de coder. L'analyse tourne sur un boîtier ou un ordinateur du site, sur le flux basse définition ; seuls les instantanés et les métadonnées sont envoyés, jamais la vidéo. N'analyse JAMAIS la vidéo en continu sur un téléphone ; PAS de reconnaissance faciale ; l'IA détecte une personne ou un véhicule, pas une intention : un « événement suspect » est défini par les règles ci-dessus.
8. Journal et conservation : chaque alerte est journalisée (heure, caméra, règle, capture, retour « utile ou inutile », qui l'a vue) ; les captures sont conservées en petite taille pendant une durée réglable (30 jours par défaut) puis supprimées automatiquement ; aucune vidéo n'est stockée chez nous ; droits par poste sur qui

reçoit quelles alertes.

9. Paliers proposés [P : à confirmer par moi] : niveau 1 dès Essentiel ; règles d'alerte avec captures au palier Pro (nombre limité de caméras) ; passerelle, regroupement avancé et IA sur place au palier Réseau.

CONTRÔLES (en local uniquement, aucune CI) : Tests avec événements simulés : règle de fermeture déclenchée, regroupement d'une rafale, alerte critique avec capture, passerelle hors ligne signalée, captures supprimées au bout de la durée réglée, aucune vidéo envoyée.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un juriste.
- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).

7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 9.8 — Recette du module Caméras

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Module « Caméras » (vidéosurveillance) à intégrer dans 6 applications de la famille : Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel (CotiZ n'a pas de lieu à surveiller). À exécuter UNIQUEMENT quand tous les prompts fonctionnels ET la refonte du design (série 8) sont terminés : l'interface reprend le design final de chaque application. Objectif : permettre au gestionnaire de voir ses caméras depuis l'application, simplement, et de relier les alertes (écart de caisse, annulation suspecte, incident) aux images. Le même module servira plus tard à l'application des écoles. Les caméras sont celles que le client possède déjà (marques courantes ou flux standard RTSP et ONVIF) ; on n'impose aucun matériel.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Contrôler la facilité, la sécurité et la performance du module dans les 6 applications.

À FAIRE :

1. Facilité : un testeur non technicien connecte une caméra seul en moins de 5 minutes dans chaque application ; liste des étapes qui bloquent.
2. Sécurité : revue des identifiants (jamais en clair, jamais dans les journaux), des accès distants (aucun port exposé), des droits par poste et du journal ; liste des risques restants.
3. Performance : 4 flux simultanés sur un téléphone d'entrée de gamme, consommation de données mesurée, comportement lors d'une coupure réseau, reconnexion automatique, arrêt des flux hors de la vue.
4. Alertes : taux d'alertes utiles mesuré sur un pilote, fausses alertes (ombres, insectes, phares), alertes manquées, délai entre l'événement et la notification, comportement quand la passerelle ou le réseau tombe ; vérification qu'aucune reconnaissance faciale n'est présente.
5. Vie privée : vérification qu'aucune caméra ne peut être placée dans un lieu d'intimité sans avertissement bloquant, rappel d'affichage, durée de conservation côté caméra ou enregistreur rappelée au client.
6. Établis un rapport avec les écarts et leur priorité. Ne publie rien.

CONTRÔLES (en local uniquement, aucune CI) : Rapport complet avec captures d'écran et mesures ; aucun test fonctionnel ne régresse.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Facilité d'abord : un assistant de connexion en moins de 5 étapes, avec un test de connexion et des messages d'erreur clairs en français et en English ; un gestionnaire non technicien doit réussir seul.
- Sécurité : jamais d'identifiant ni de mot de passe de caméra en clair dans Supabase ni dans les journaux ; stockage chiffré sur l'appareil (Android Keystore) ; ne JAMAIS ouvrir un port de caméra directement sur Internet ; accès distant uniquement par la marque, un relais sécurisé ou un tunnel gratuit à jetons de courte durée.
- Vie privée et droit : aucune caméra dans les lieux d'intimité (toilettes, vestiaires, chambres d'hôtel) ; droits d'accès par poste ; chaque consultation est journalisée (qui, quand, quelle caméra) ; l'application rappelle l'obligation d'informer les personnes filmées (affichage) ; règles légales camerounaises à faire valider par un

juriste.

- Performance et données mobiles : au plus 4 flux simultanés, flux secondaire basse définition par défaut, Wi-Fi par défaut avec avertissement avant d'utiliser les données mobiles (consommation estimée en Mo par heure), arrêt automatique du flux quand l'écran quitte la vue.
- Le module doit être optionnel et ne jamais ralentir ni casser le reste de l'application ; il suit le design final de chaque application (série 8).

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »